

PROJECTE OPCIONAL: K-means clustering paral·lel

Explicació del projecte: Objectius i enfocament

K-means és un algoritme de Clustering (Agrupament) amb aprenentatge no supervisat utilitzat al camp de la Intel·ligència artificial per realitzar problemes de classificació.

K-means conté una estructura per iteracions, on cada iteració conté dos fases:

1- **Assignació** → per cada punt p trobem el centre més proper i el guardem independentment.

2- **Actualització de centroides** → per cada grup/clúster k sumem les contribucions.

L'objectiu principal del projecte és estudiar i avaluar el paral·lisme de l'algoritme en una arquitectura de memòria compartida, analitzant com la realització dels petits canvis amb la paralelització canvien el rendiment sobre el temps d'execució, mesurant i analitzant les mètriques com el speed-up, eficiència, escalabilitat...

Busquem analitzar el comportament del algoritme desde el punt de vista del paral·lisme, tal i com hem realitzat a classe.

Versions realitzades

Per realitzar els anàlisis i comparacions de diferents estratègies de paral·lelització farem servir diferents versions:

1. Seqüencial

Codi inicial i base per validar resultats i calcular el temps base de l'algoritme.

Explicació del codi

Inicialitzem els acumuladors (sx, sy, ct) per clúster:

```
C/C++
double *sx = calloc(K, sizeof(double)); // guardem la suma de coordenades
dels punts assignats als clúster k
double *sy = calloc(K, sizeof(double));
int *ct = calloc(K, sizeof(int)); // contador dels punts que conté el
clúster k
```

Calculem distàncies desde cada punt a cada clúster per trobar el més proper:

```
C/C++
for(int i=0;i<N;i++){ // per cada punt cerquem el centre més proper
    int best=0;
    float bestd = dist2(P[i], C[0]);
    for(int k=1;k<K;k++){
        float d = dist2(P[i], C[k]);
        if(d<bestd){ bestd=d; best=k; }
    }
    label[i]=best; // guardem l'assignació del clúster escollit que està a
menys distància del punt
    sx[best]+=P[i].x;
    sy[best]+=P[i].y;
    ct[best]+=1;
}
```

Recalculem centroides:

```
C/C++
for(int k=0;k<K;k++){
    if(ct[k]>0){
        C[k].x = (float)(sx[k]/ct[k]); // sumem totes les posicions en x, y
per dividir entre tots els punts d'aquest mateix clúster
        C[k].y = (float)(sy[k]/ct[k]);
    }
}
```

2. Paral·lela

Utilització **atòmics** per protegir les actualitzacions dels centroides (centroide = punt representatiu del grup calculat amb la mitjana de la posició dels punts d'aquest grup)
→ Al bucle de punts actualitzar la variable **sum[k]** i **count[k]** amb **#pragma omp atomic** (també es podria realitzar un critical).

Explicació del codi

Amb aquest codi intentarem paral·lelitzar l'algoritme de la forma més directa possible, creant paral·lelisme sobre els punts calculats amb la protecció de les actualitzacions utilitzant operacions atòmiques i mantenint un únic conjunt de centroides compartits.

1. Explicació dels paràmetres de la funció

C/C++

```
void kmeans_omp_atomic(const point2d *P, int N, point2d *C, int K, int iters,
int *label)
```

Tenim els punts de les dades, el número de punts per evaluar, els centroides, nombre de clústers/grups, iteracions a realitzar, assignació del punt a un clúster amb label.

2. Reserva d'acumuladors compartits

C/C++

```
double *sx = malloc(K * sizeof(double));
double *sy = malloc(K * sizeof(double));
int *ct = malloc(K * sizeof(int));
```

3. Realitzem unes quantes iteracions (per a que l'algoritme K-means pugui convergir) i inicialitzem a 0 els acumuladors de forma seqüencial.

C/C++

```
for(int it=0; it<iters; ++it){

    for(int k=0; k<K; k++){
        sx[k]=0.0;
        sy[k]=0.0;
        ct[k]=0;
    }
}
```

Es realitza fora de la regió paral·lela perquè sol ser un cost petit (K petit).

4. Assignació de punts a la regió paral·lela

C/C++

```
#pragma omp parallel for schedule(static)
for(int i=0; i<N; i++){
```

Dividim el rang entre els threads en blocs fixos gràcies a “for schedule(static)” que sempre és determinista (útil quan tenim un treball uniforme entre operacions). Cada fil processa un subconjunt de punts.

5. Cerca del clúster adient mitjançant el càlcul de la distància al centroide més proper i actualitzacions de centroides amb **atomic**

```
C/C++
#pragma omp parallel for schedule(static)
for(int i=0;i<N;i++){
    int best = 0;
    float bestd = dist2(P[i], C[0]);
    for(int k=1;k<K;k++){
        float d = dist2(P[i], C[k]);
        if(d < bestd){ bestd = d; best = k; }
    }
    label[i] = best;

    #pragma omp atomic
    sx[best] += (double)P[i].x;

    #pragma omp atomic
    sy[best] += (double)P[i].y;

    #pragma omp atomic
    ct[best] += 1;
}
```

En aquest punt observem un coll d'ampolla molt important. Sabem que les operacions s'executen de forma independent, ja que OpenMP ens assegura una exclusió mútua de les operacions amb atomic. En canvi, si molts threads volen actualitzar la mateixa posició del array ens trobem amb un problema de tràfic de coherència com a conseqüència de les esperes actives que estan realitzant la resta dels threads que no han pogut accedir a la variable. Tenim un problema de **true sharing**, on a la mateixa variable volen escriure diferents threads alhora i acumula un cost de sincronització innecessari.

Es pot observar que no és gens escalable, per exemple: si $N=2000000$, iteracions = 20, operacions atomic = 120000000

3. Paral·lela optimitzada

Cada thread conté arrays locals amb els paràmetres processant el seu bloc de punts i realitzar una única actualització per reduir coherència.

Explicació del codi

Com ja hem comentat anteriorment, si els threads intenten accedir a la mateixa posició de la variable per realitzar actualitzacions o assignar valors (per exemple: `sx[best] += ...`) tenim data races.

Amb els mecanismes de sincronització que ens proporciona OpenMP podem evitar aquests problemes, però hem de tenir en compte els colls d'ampolla si utilitzem atomic a cada punt.

Farem una assignació manual dels threads:

1. Creem regió paral·lela i calculem quants threads tenim per realitzar la paral·lelització:

```
C/C++
int T = 0;

#pragma omp parallel
{
    #pragma omp single
    T = omp_get_num_threads();
}
```

2. Reservem acumuladors per cada thread (cada fil té el seu propi array de K posicions)

```
C/C++
double *sx_all = calloc((size_t)T*K, sizeof(double));
double *sy_all = calloc((size_t)T*K, sizeof(double));
int *ct_all = calloc((size_t)T*K, sizeof(int));
```

Evitem que els fils puguin escriure a la mateixa casella al bucle principal.

3. Realitzem unes quantes iteracions (per a que l'algoritme K-means pugui convergir) i inicialitzem a 0 els acumuladors de cada thread (cada thread escriu independentment)

```
C/C++
for(int it=0; it<iters; ++it){

    // zero locals
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for(int k=0; k<K; k++){
            sx_all[tid*K+k]=0.0;
            sy_all[tid*K+k]=0.0;
            ct_all[tid*K+k]=0;
        }
    }
}
```

4. Repartim manualment cada punt mitjançant blocs → **Block decomposition**

```
C/C++
int nth = omp_get_num_threads();
int bs = (N + nth - 1)/nth;
int i0 = tid*bs;
int i1 = (i0+bs < N) ? (i0+bs) : N;
```

5. Assignació i acumulació local de grups

```
C/C++
for(int i=i0;i<i1;i++){
    int best=0;
    float bestd = dist2(P[i], C[0]);
    for(int k=1;k<K;k++){
        float d = dist2(P[i], C[k]);
        if(d<bestd){ bestd=d; best=k; }
    }
    label[i]=best;
    sx_all[tid*K+best] += P[i].x;
    sy_all[tid*K+best] += P[i].y;
    ct_all[tid*K+best] += 1;
}
```

Accedim a la posició dels arrays mitjançant el id del thread per aconseguir un accés local de cada thread, on cadascun té el seu propi array de K posicions. No ens fa falta atomic.

6. Reducció final: actualitzem centroides amb les contribucions de cada thread per cada clúster

```
C/C++
for(int k=0;k<K;k++){
    double sx=0, sy=0; int ct=0;
    for(int tid=0; tid<T; tid++){
        sx += sx_all[tid*K+k];
        sy += sy_all[tid*K+k];
        ct += ct_all[tid*K+k];
    }
    if(ct>0){
        C[k].x = (float)(sx/ct);
        C[k].y = (float)(sy/ct);
    }
}
```

Metodologia i Resultats de l'avaluació

En aquest apartat realitzarem diferents validacions per tal de representar les millores obtingudes gràcies a l'estratègia de paral·lelització realitzada.

Començarem amb una avaluació qualitativa observant una execució amb un problema de classificació petit ($N < 500000$). I continuem amb una avaluació basada en strong scaling, on cada versió farem servir experiments amb tamany fix i nombre de threads variables mesurant el temps d'execució, speed-up i eficiència.

1. Validació funcional

Shell

```
paco1114@boada-11:~/Proj_Opc$ gcc -O3 -march=native -fopenmp kmeans.c -o kmeans
paco1114@boada-11:~/Proj_Opc$ ./kmeans 100000 8 5 123 seq
N=100000 K=8 iters=5 seed=123 mode=seq threads=1 time=0.008696
checksum=464889973478590237

paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=4 ./kmeans 100000 8 5 123 atomic
N=100000 K=8 iters=5 seed=123 mode=atomic threads=4 time=0.016090
checksum=464889973478590237

paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=4 ./kmeans 100000 8 5 123 locals
N=100000 K=8 iters=5 seed=123 mode=locals threads=4 time=0.009092
checksum=464889973478590237
```

Abans d'analitzar el rendiment, es valida la correcció de les implementacions. Per a una mateixa configuració d'entrada (N, K, nombre d'iteracions i seed), totes les versions produeixen el mateix checksum, fet que indica que la paral·lelització no introdueix errors de concurrència ni data races.

Però podem veure que amb una mida de problema petita ($N=100.000$), la versió paral·lela amb atomic presenta un temps d'execució superior a la versió seqüencial. Aquest comportament indica que el cost de sincronització i de gestió del paral·lisme domina el temps de còmput quan la càrrega de treball és petita.

2. Strong Scaling

→ Utilitzem un problema amb $N = 2\,000\,000$ de punts a classificar, 32 clústers, 20 iteracions, seed = 123

2.1. Seqüencial

Shell

```
paco1114@boada-11:~/Proj_Opc$ ./kmeans 2000000 32 20 123 seq
N=2000000 K=32 iters=20 seed=123 mode=seq threads=1 time=2.179880
checksum=8221321130988373822
```

2.2. Atomic

Shell

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=1 ./kmeans 2000000 32 20 123
atomic
N=2000000 K=32 iters=20 seed=123 mode=atomic threads=1 time=2.872492
checksum=8221321130988373822
```

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=2 ./kmeans 2000000 32 20 123
atomic
N=2000000 K=32 iters=20 seed=123 mode=atomic threads=2 time=2.995801
checksum=8221321130988373822
```

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=4 ./kmeans 2000000 32 20 123
atomic
N=2000000 K=32 iters=20 seed=123 mode=atomic threads=4 time=2.846025
checksum=8221321130988373822
```

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=8 ./kmeans 2000000 32 20 123
atomic
N=2000000 K=32 iters=20 seed=123 mode=atomic threads=8 time=2.875119
checksum=8221321130988373822
```

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=16 ./kmeans 2000000 32 20 123
atomic
N=2000000 K=32 iters=20 seed=123 mode=atomic threads=16 time=2.890965
checksum=8221321130988373822
```


2.3. Locals

Shell

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=1 ./kmeans 2000000 32 20 123
```

```
locals
```

```
N=2000000 K=32 iters=20 seed=123 mode=locals threads=1 time=2.197283
```

```
checksum=8221321130988373822
```

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=2 ./kmeans 2000000 32 20 123
```

```
locals
```

```
N=2000000 K=32 iters=20 seed=123 mode=locals threads=2 time=2.296968
```

```
checksum=8221321130988373822
```

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=4 ./kmeans 2000000 32 20 123
```

```
locals
```

```
N=2000000 K=32 iters=20 seed=123 mode=locals threads=4 time=2.197288
```

```
checksum=8221321130988373822
```

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=8 ./kmeans 2000000 32 20 123
```

```
locals
```

```
N=2000000 K=32 iters=20 seed=123 mode=locals threads=8 time=2.217487
```

```
checksum=8221321130988373822
```

```
paco1114@boada-11:~/Proj_Opc$ OMP_NUM_THREADS=16 ./kmeans 2000000 32 20 123
```

```
locals
```

```
N=2000000 K=32 iters=20 seed=123 mode=locals threads=16 time=2.193855
```

```
checksum=8221321130988373822
```

Prova realista - Strong Scaling

L'anterior test no és gaire realista perquè hem utilitzat una quantitat de punts molt petita i no permeten visualitzar les millores de rendiment entre les diferents versions de paral·lelització. Per tant, hem construït un script .sh per realitzar una nova prova amb 10 milions de punts a classificar, 32 clústers, 20 iteracions i 123 de Seed, per tal de poder veure el rendiment real i les diferències entre totes les versions de codi.

Es pot observar com totes les execucions contenen el mateix valor de checksum, confirmant així que les tres implementacions són correctes.

Resultat de seqüencial

```
None
paco1114@boada-11:~/Proj_Opc$ cat results_seq.txt

kmeans_seq_10M_388308.out:N=10000000 K=32 iters=20 seed=123 mode=seq threads=1
time=7.952001 checksum=3587576609566897937
```

Aquest resultat de la versió seqüencial l'utilitzem com a referència pel càlcul del Speed-Up global.

Resultat de atomic

```
None
paco1114@boada-11:~/Proj_Opc$ cat results_atomic.txt
kmeans_atomic_10M_388307.out:N=10000000 K=32 iters=20 seed=123 mode=atomic
threads=1 time=10.874348 checksum=3587576609566897937

kmeans_atomic_10M_388307.out:N=10000000 K=32 iters=20 seed=123 mode=atomic
threads=2 time=13.704727 checksum=3587576609566897937
kmeans_atomic_10M_388307.out:N=10000000 K=32 iters=20 seed=123 mode=atomic
threads=4 time=22.677429 checksum=3587576609566897937

kmeans_atomic_10M_388307.out:N=10000000 K=32 iters=20 seed=123 mode=atomic
threads=8 time=20.241117 checksum=3587576609566897937

kmeans_atomic_10M_388307.out:N=10000000 K=32 iters=20 seed=123 mode=atomic
threads=16 time=16.484115 checksum=3587576609566897937
```

Fils (p)	Temps T_p (s)	Speed-up $S_p = T_i/T_p$	Eficiència $E_p = S_p/p$
1	10.874348	0.73	0.73
2	13.704727	0.58	0.29
4	22.677429	0.35	0.09
8	20.241117	0.39	0.05
16	16.484115	0.48	0.03

Com podem veure, aquesta paral·lelització no és gens bona, ja que la sincronització molt fina amb les operacions atòmiques, la contenció d'acumuladors compartits i el tràfic de

coherència anulen per complet els beneficis que obtenim del paral·lelisme. No és una versió escalable, sinó que contraproductent.

Resultat de locals

None

```
paco1114@boada-11:~/Proj_Opc$ cat results_locals.txt
```

```
N=10000000 K=32 iters=20 seed=123 mode=locals threads=1 time=7.956104  
checksum=3587576609566897937
```

```
N=10000000 K=32 iters=20 seed=123 mode=locals threads=2 time=5.743497  
checksum=3587576609566897937
```

```
N=10000000 K=32 iters=20 seed=123 mode=locals threads=4 time=4.556420  
checksum=3587576609566897937
```

```
N=10000000 K=32 iters=20 seed=123 mode=locals threads=8 time=2.526500  
checksum=3587576609566897937
```

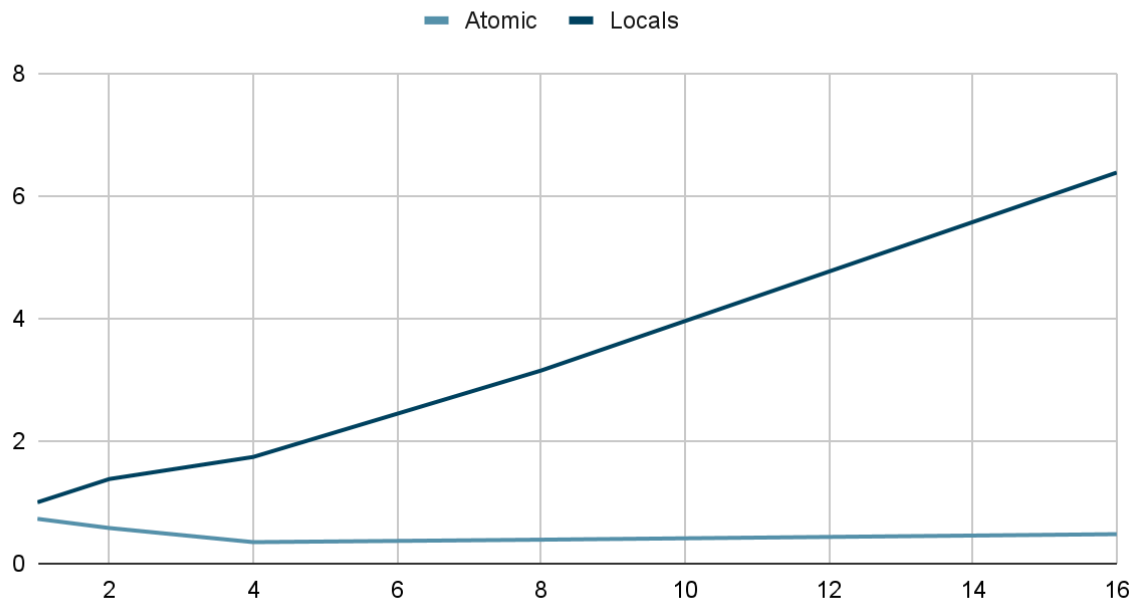
```
N=10000000 K=32 iters=20 seed=123 mode=locals threads=16 time=1.246877  
checksum=3587576609566897937
```

Fils (p)	Temps T_p (s)	Speed-up $S_p = T_1/T_p$	Eficiència $E_p = S_p/p$
1	7.956104	1.00	1.00
2	5.743497	1.38	0.69
4	4.556420	1.74	0.44
8	2.526500	3.15	0.39
16	1.246877	6.38	0.40

La versió amb acumuladors locals escala correctament quan el problema és prou gran, demostrant que reduir la sincronització fina permet aprofitar el paral·lelisme, mentre que per problemes petits l'overhead domina.

Gràfic del Speedup Atomic vs Locals

SPEEDUP RESULTS



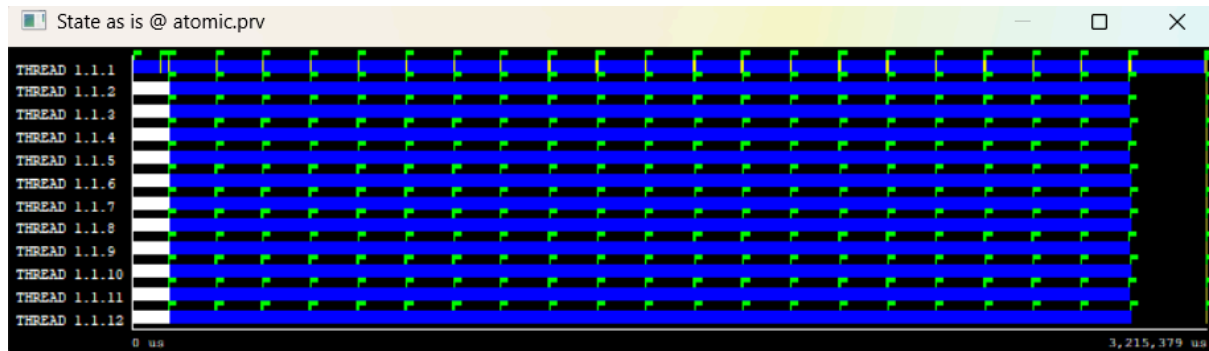
El gràfic il·lustra clarament l'impacte de l'estratègia de paral·lelització en el rendiment.

- **Versió Atomic (Línia clara):** Mostra una **escalabilitat negativa** (Speedup < 1). La línia es manté plana o descendent en afegir fils, indicant que el sistema està **saturat**. L'overhead de la sincronització atòmica i la contenció pel bus de memòria anul·len qualsevol benefici del paral·lelisme.
- **Versió Locals (Línia fosca):** Presenta una **escalabilitat positiva**, arribant a un Speedup de **6.38x** amb 16 fils. El pendent ascendent confirma que l'ús de memòria privada (privatització) elimina el coll d'ampolla de l'accés a memòria, permetent que els fils treballin de manera eficient.

3. Anàlisi Visual de l'Execució (Paraver)

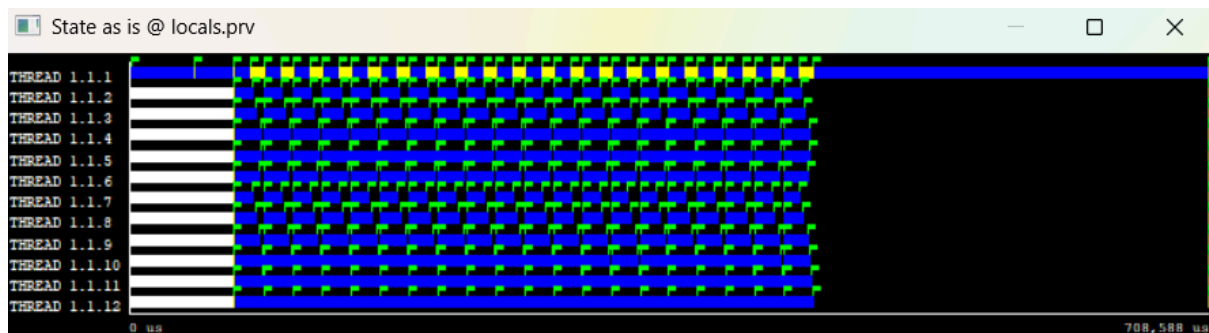
Per validar les mètriques obtingudes, hem analitzat les traces d'execució amb wxParaver per veure el comportament real dels fils en el temps.

Figura 1: Traça de la versió ATOMIC:



La imatge mostra una gran fragmentació en la línia temporal (barres trencades). Això visualitza la **contenció**: els fils passen més temps esperant el *lock* atòmic que processant dades. El temps total és molt elevat (~3.2s).

Figura 2: Traça de la versió LOCALS:



S'observen blocs de color continu (blau = running). Gràcies a l'ús de variables privades, els fils treballen al 100% de capacitat sense molestar-se entre ells. La sincronització només passa al final de la iteració (marques grogues). El temps baixa dràsticament (~0.7s).

Nota: No ha calgut utilitzar Tareador perquè l'anàlisi de traces amb Paraver (versió Locals) demostra visualment l'absència de dependències de dades.

Conclusions

En aquest treball s'ha estudiat la paral·lelització de l'algorisme K-means clustering, un mètode clàssic d'aprenentatge no supervisat, utilitzant el model de memòria compartida amb OpenMP. L'objectiu principal ha estat analitzar com diferents estratègies de paral·lelització afecten el rendiment real de l'algorisme, posant especial èmfasi en els costos associats a la sincronització i a la compartició de dades.

S'han implementat tres versions del codi: una versió seqüencial de referència, una versió paral·lela naïf basada en operacions atòmiques i una versió paral·lela optimitzada amb acumuladors locals per fil. Aquesta estructura ha permès comparar de manera directa l'impacte de les decisions de disseny en el rendiment i la seva escalabilitat.

Els resultats experimentals mostren de manera clara que la versió paral·lela basada en operacions atòmiques no és una estratègia adequada per a aquest tipus d'algorismes. Tot i garantir la correcció, la sincronització fina introduïda per les operacions atòmiques genera una elevada contenció sobre les dades compartides i un trànsit intens de coherència de memòria. Com a conseqüència, el temps d'execució augmenta en lloc de disminuir, i el speed-up obtingut és inferior a la unitat per a tots els nombres de fils analitzats. Aquest comportament posa de manifest que una paral·lelització directa i no acurada pot resultar contraproductiu.

En canvi, la versió paral·lela amb acumuladors locals presenta un comportament substancialment millor quan el problema té una mida suficientment gran. En eliminar les actualitzacions concurrents sobre dades compartides i relegar la sincronització a una fase de reducció final, s'aconsegueix reduir significativament l'overhead associat al paral·lelisme. Per a $N = 10.000.000$ punts, aquesta versió aconsegueix un speed-up de fins a 6,38 amb 16 fils, amb una eficiència propera al 40%, fet que evidencia una explotació efectiva del paral·lelisme disponible.

Un aspecte rellevant observat és que, per mides de problema més petites, fins i tot la versió optimitzada no aconsegueix millores significatives respecte a la versió seqüencial. Això indica que el cost de gestió del paral·lelisme i la presència de fraccions seqüencials poden dominar el temps total quan la càrrega de treball no és suficient per amortitzar aquests costos. Aquest fet és coherent amb els límits teòrics del speed-up i amb els conceptes d'overhead i fracció no paral·lelitzable estudiats a l'assignatura.

En conjunt, aquest treball posa de manifest que paral·lelitzar no consisteix únicament a afegir fils d'execució, sinó que requereix un disseny acurat de l'estratègia de compartició de dades i sincronització. La comparació entre les versions atòmiques i locals exemplifica clarament com una bona elecció de la granularitat i la reducció de la sincronització poden marcar la diferència entre una implementació ineficient i una que escala correctament.

Finalment, els resultats obtinguts reforcen la importància d'analitzar experimentalment els algorismes paral·lels i d'utilitzar eines i mètriques adequades per comprendre el seu comportament real. El treball realitzat permet connectar de manera directa la teoria del paral·lelisme amb una aplicació pràctica d'intel·ligència artificial, complint així els objectius

plantejats i aportant una visió clara i crítica sobre els factors que condicionen el rendiment en sistemes de memòria compartida.